

MIRAGE-X

Integrated Adaptive Cyber Defense & Attacker Intelligence Platform

A complete project overview for sharing with collaborators, judges, mentors, and future contributors.

Project purpose. MIRAGE-X is a local, simulation-first cyber defense MVP that detects suspicious behavior, scores and explains it, predicts an attacker's next target, deploys a behavior-matched decoy, learns from the interaction, and presents the full story in a dashboard.

Core differentiator. The system is a closed loop rather than a static alert screen: **sensor/simulator -> Attack DNA -> memory and confidence -> adaptive deception -> evidence feedback -> recomputation -> explainable dashboard and SOC alerting.**

At a glance

Area	Current implementation
Backend	FastAPI service with SQLite persistence and 20+ REST operations.
Detection	Rule-based behavior sequences, risk scoring, confidence bands, and false-positive gates.
Memory	Explainable similarity using sequence, MITRE technique overlap, and risk-band matching.
Deception	Fake SSH, fake database, and fake admin panel decoys with interaction logging.
Intelligence	Attack-path prediction, MITRE ATT&CK; mapping, SOC severity, and grounded advisor.
Frontend	Responsive static dashboard with replay, trend, comparison, alerts, settings, onboarding, and accessibility.
Validation	30 automated tests pass; Python syntax compilation passes.
Safety	Simulation-first design, no real credentials, isolated Docker decoys, no live response automation.

1. Product vision and architecture

MIRAGE-X turns individual security events into an adaptive investigation workflow. Instead of treating every event as an isolated alert, it builds a behavioral fingerprint, compares that fingerprint with prior incidents, selects an appropriate decoy only when justified, and feeds decoy evidence back into the incident model.

Closed-loop data flow

Stage	What happens	Primary modules
1. Observe	A repeatable scenario or API event enters the system.	simulator.py, POST /ingest
2. Fingerprint	Events become a behavior sequence and risk score.	attack_dna.py
3. Remember	Current behavior is compared with stored incidents.	memory.py, db.py
4. Decide	Risk band and similarity drive decoy selection.	deception.py
5. Predict	Behavior is mapped to a likely next network target.	attack_path.py
6. Enrich	MITRE techniques and SOC severity are attached.	mitre.py, soc_alert.py
7. Deceive	The selected isolated decoy records raw interaction evidence.	decoys/*, common.py
8. Learn	Evidence is folded into events and the pipeline recomputes.	decoy_feedback.py, POST /recompute
9. Explain	Dashboard, PDF, alerts, and advisor present grounded facts.	frontend, reports.py, advisor.py

Repository structure

- **backend/**: FastAPI application, deterministic engines, SQLite helpers, decoys, and tests.
- **frontend/**: Static HTML/CSS/JavaScript dashboard; no frontend build step is required.
- **docker/**: Dockerfiles for the three isolated decoy services.
- **docs/**: Original concept PDF, build guide, and this shareable overview.
- **docker-compose.yml**: Runs fake SSH, fake DB, and fake admin panel on an internal Docker network.
- **reset_demo.sh**: Resets demo state for repeatable presentations.

2. Implemented backend capabilities

Capability	Implemented behavior
FastAPI service	Typed request models, CORS support, startup database initialization, HTTP errors, interactive OpenAPI docs.
SQLite persistence	Stores incidents, ordered events, decoy interactions, folded state, and simulated SOC alerts.
Scenario simulator	Ten deterministic scenarios cover normal administration and multiple realistic attack patterns.
Event ingestion	Accepts external event records with incident ID, event type, source IP, host, user, and detail.
Risk engine	Calculates behavior sequence, numeric risk score, and risk band from event evidence.
False-positive controls	Benign admin activity remains below the medium-risk threshold and does not trigger a decoy.
Incident memory	Finds the closest prior incident and exposes matched features and similarity score.
Adaptive deception	Routes database hunting, lateral movement, SSH abuse, privilege escalation, and admin activity to matching decoys.
Explainability	Produces a human-readable summary from observed evidence, not an opaque model decision.

3. Implemented intelligence and defense layers

Attack DNA and risk scoring

The deterministic Attack DNA layer converts raw events into an ordered behavior sequence. It assigns risk from event types and sequence-level bonuses, then maps the score to a confidence/risk band. The implementation covers 14 event types, including reconnaissance, SSH authentication, privilege escalation, database search, credential spraying, lateral movement, exfiltration staging and transfer, supply-chain compromise, living-off-the-land activity, insider access, administrative action, and decoy-derived evidence.

Cyber Memory

- Sequence similarity recognizes behavior even when source indicators differ.
- Technique overlap compares the MITRE techniques implied by two sequences.
- Risk-band agreement adds context without making similarity alone a threat verdict.
- The result is surfaced as a top match with an explainable score and matched features.

Attack-Path and Intent Engine

- Maintains a simulated seven-node network graph.
- Predicts a likely next target from the observed behavior sequence.
- Includes a second host for lateral movement and an external egress point for exfiltration.
- Exposes both a JSON prediction and a graph snapshot used by the SVG dashboard diagram.

MITRE ATT&CK; mapping

Every supported event type maps to a real technique ID, name, and tactic. Examples include T1110.003 Password Spraying, T1560 Archive Collected Data, T1048 Exfiltration Over Alternative Protocol, and T1595 Active Scanning. Technique tags are returned by the API, shown in the dashboard, and included in generated incident reports.

SOC severity simulation

The SOC layer assigns CRITICAL, HIGH, MEDIUM, LOW, or INFO severity and a would_page flag. High-impact patterns such as exfiltration and supply-chain signals can page even when a raw numeric score alone would not be sufficient. The alert channel stores a Slack-styled simulated message, deduplicates repeats, and only re-fires on a genuine escalation.

Adaptive deception and feedback

Three decoys are implemented as real socket/HTTP services: fake SSH on port 2222, fake DB on port 3307, and fake admin panel on port 8080. They do not provide real credentials or real data. Raw traffic is classified into clean evidence such as CLIENT_BANNER, PROBE_ONLY, and BRUTEFORCE_PATTERN_DETECTED. The feedback loop folds new evidence into proper incident events idempotently, then recomputes risk, path prediction, memory, and deception.

4. Implemented frontend and user experience

The frontend is a static dashboard in `frontend/index.html`, `frontend/style.css`, and `frontend/script.js`. It can be opened directly in a browser and currently targets the deployed backend URL configured at the top of `script.js`.

Dashboard feature	What the user can do
Scenario runner	Choose and execute any simulator scenario, reset the demo, and watch the result.
Incident list	Search by incident ID or behavior, filter by risk level and decoy, and select an incident.
Risk and timeline	See risk score, band, ordered events, evidence, summary, and a step-by-step risk trend.
Replay scrubber	Play, pause, and scrub through event arrival to narrate how risk escalated.
Attack-path diagram	See the simulated graph with current and predicted next nodes highlighted.
Memory view	Inspect the closest historical match, similarity, and matched features.
MITRE and SOC panels	Read technique tags, severity, page decision, and simulated alert reasoning.
Decoy evidence	Review interactions captured by fake SSH, fake DB, and fake admin panel.
Feedback action	Fold decoy evidence into the incident and recompute the analysis.
PDF report	Download a per-incident PDF containing facts, techniques, alert data, and advisor content.
AI Advisor	Read a grounded summary, recommended actions, and ask a question about the incident.
Correlation	Ask the advisor to compare all incidents currently held in memory.
Compare	Select two incidents for a side-by-side risk, decoy, technique, and path comparison.
Alerts	Open the simulated SOC channel, see unread counts, and receive toast notifications.
Settings	Toggle beginner explanations, panel visibility, theme, accent color, and per-request API key.
Onboarding	Use a five-slide guided tour covering the main demo flow and dashboard controls.
Accessibility	Use skip navigation, keyboard focus, modal semantics, Escape handling, live regions, and visible focus outlines.
Responsive layout	Use the dashboard on a projector, desktop, tablet, or narrow phone-sized viewport.

AI Advisor design

The advisor is deliberately placed above the deterministic engine. It never re-decides risk, the selected decoy, MITRE mapping, or SOC severity. With an Anthropic key it can produce live explanations; without a key, or when a call fails, it uses a rule-based fallback built from the same facts. Transient timeouts, connection issues, and rate limits receive bounded retries. The dashboard distinguishes live AI, missing key, invalid key, and temporary unavailability rather than hiding the cause.

5. Scenarios and API surface

Built-in scenarios

Scenario	Purpose and notable behavior
recon_to_db_hunt	Classic kill chain: RECON-SCAN -> AUTH-SSH -> PRIV-ESC -> DOWNLOAD -> DB-SEARCH.
recon_to_db_hunt_variant_ip	Same behavior from a different source, demonstrating behavior-first memory matching.
ssh_bruteforce_only	Repeated SSH authentication attempts against one username.
benign_admin_login	Expected administrative activity used as the false-positive control.
password_spray_campaign	Many usernames with one password each; emits CRED-SPRAY.
lateral_movement_breach	Pivots to a second host and reaches database search.
data_exfiltration	Stages data and transfers it out; produces a CRITICAL page.
supply_chain_compromise	Poisoned dependency or CI step without human login events.
living_off_the_land	Legitimate tools used in a suspicious sequence.
insider_misuse	Valid credentials combined with scope drift and sensitive data access.

REST endpoints

Method	Endpoint	Purpose
GET	/scenarios	List simulator scenarios.
POST	/simulate	Run a scenario and process its incident.
POST	/ingest	Insert one event and recompute the incident.
GET	/incidents	List stored incidents.
GET	/incidents/{id}	Return a fully enriched incident.
GET	/network	Return the simulated network graph.
GET	/incidents/{id}/predicted_path	Return next-target prediction.
GET	/incidents/{id}/risk_trend	Return step-by-step risk points.
GET	/incidents/{id}/similar	Return closest memory match.
POST	/incidents/{id}/recompute	Fold decoy evidence and recompute.
GET	/alerts	List simulated SOC alerts.
POST	/reset	Reset the demo database.
GET	/incidents/{id}/report	Download a generated PDF report.
GET	/settings/llm_status	Check whether an advisor key is available.
GET	/incidents/{id}/advisory	Generate grounded advisory.
POST	/incidents/{id}/ask	Ask a question about one incident.
POST	/advisor/correlate	Correlate all stored incidents.
GET	/health	Health status, database check, version, and uptime.
GET	/events/stream	Server-Sent Events for incidents, decoy evidence, and SOC alerts.
GET	/analytics	Current-memory risk, MITRE, decoy, false-positive, and SOC aggregates.

6. Deployment, safety, and validation

Local run

Install backend/requirements.txt, initialize the SQLite database, start uvicorn on port 8000, and open frontend/index.html. The backend exposes interactive API documentation at /docs. The frontend requires no package manager or build step. Pass `?api=http://localhost:8000` to the dashboard when the backend is not at its default URL.

Render deployment

- **render.yaml** defines the Python web service, `/health` health check, uvicorn start command, and a 1 GB persistent disk mounted at /var/data`.`
- Set **MIRAGE_DB_PATH=/var/data/mirage_x.db** so incident memory survives restarts.
- Set **MIRAGE_ALLOWED_ORIGINS** to the exact dashboard origins; use **ANTHROPIC_API_KEY** only when live advisor calls are desired.
- The deployed dashboard can be pointed at the service with `?api=<backend-url>`; cold-start retries and an unreachable banner are built in.

Dockerized decoys

- docker-compose.yml builds all three decoy services.
- The services share the backend code and SQLite file through a bind mount, so live evidence appears in the same dashboard.
- Ports 2222, 3307, and 8080 are published for controlled local demonstration.
- The mirage-isolated Docker network is internal, preventing outbound internet access from the decoy containers.
- The design supports nmap, hydra, netcat, curl, and browser demonstrations in a lab environment.

Safety boundary

- This is a simulation and deception lab, not a tool for attacking real systems.
- Decoys contain no real credentials, production secrets, or real business data.
- The SOC channel is explicitly simulated; it does not post to a real Slack or messaging workspace.
- The advisor explains and recommends; it does not execute remediation or alter real infrastructure.
- Any live-tool testing should stay on localhost or an explicitly isolated lab network.

Validation completed

Check	Result
Python syntax compilation	Passed with <code>python -m compileall -q backend</code> .
Automated backend tests	12 passed, covering false positives, new scenarios, routing, SOC paging, and MITRE tags.
Repository state	The reviewed branch was clean and had no uncommitted changes.
Known warning	FastAPI reports a non-blocking deprecation warning for <code>on_event</code> startup handlers.
Deployment check	Render metadata is included; the configured public URL should be rechecked before sharing a live demo link.

7. Recommended implementation roadmap

The following additions are ordered by practical value. They preserve the current deterministic and safety-first design rather than replacing it with an ungrounded AI system.

Priority	Further implementation	Expected value
P0	Make deployment reliable: health checks, startup migration, persistent database, logs, and a monitored public URL.	Ensures the shared dashboard works for every reviewer.
P0	Add end-to-end tests for all ten scenarios and API contract tests for every endpoint.	Catches integration regressions beyond unit-level behavior.
P0	Replace permissive CORS in production with an explicit frontend origin allowlist.	Reduces unnecessary cross-origin exposure.
P1	WebSocket or Server-Sent Events updates instead of four-second polling.	Makes live decoy evidence and alerts feel immediate.
P1	Historical analytics view across incidents: trends, top techniques, decoy yield, and false-positive rate.	Turns the demo into an operational intelligence product.
P1	Authentication and role-based access for analysts, viewers, and demo administrators.	Supports safe multi-user deployment.
P1	Background job queue for report generation and advisor requests.	Prevents slow PDF or AI work from blocking API requests.
P1	Configurable rules and scoring profiles stored outside source code.	Allows different organizations to tune risk without code changes.
P2	OpenTelemetry metrics, structured logs, and audit trail for every decision.	Improves observability and makes decisions defensible.
P2	Replayable event stream and export/import of sanitized incidents.	Enables reproducible research, training, and offline demos.
P2	Pluggable sensors for Zeek, Suricata, Windows events, cloud audit logs, or SIEM webhooks.	Moves from simulation toward controlled real telemetry ingestion.
P2	Decoy quality measurement: dwell time, interaction depth, and attacker-path divergence.	Quantifies whether deception is useful, not merely present.
P2	Human feedback workflow for analyst verdicts and rule tuning.	Creates a safe path toward supervised learning.
P3	Advanced sequence models or graph analytics behind the deterministic safety gate.	Adds research value while preserving explainable fallback behavior.
P3	Multi-tenant storage, retention policies, encryption, and secret management.	Prepares the platform for production-grade deployment.

8. Suggested demonstration script

A reliable 90-second presentation can show the complete loop without needing real external infrastructure.

Step	Demonstration
1	Reset the demo and show an empty incident list.
2	Run recon_to_db_hunt. Explain the sequence, score, memory result, predicted path, and fake DB decision.
3	Open the timeline replay and scrub through events to show risk climbing.
4	Send controlled nmap, hydra, netcat, or curl traffic to a local decoy and show classified evidence.
5	Fold the evidence into the incident and point out the recomputed risk and predicted target.
6	Run recon_to_db_hunt_variant_ip and show that behavior similarity works despite a different source.
7	Run data_exfiltration and show CRITICAL severity, paging, MITRE techniques, and the downloadable PDF.
8	Use Compare or Cross-Incident Correlation to demonstrate reasoning across incidents.
9	Open Settings to show Beginner Mode, theme, responsive controls, and optional grounded AI Advisor configuration.

9. Final status

MIRAGE-X is a strong working MVP and demonstration platform. Its most complete story is the deterministic closed loop: observe behavior, understand it, predict the next step, deploy a safe decoy, learn from interaction, and explain the result. The immediate work is reliability and integration testing rather than adding unbounded features. The strongest future direction is to connect sanitized real telemetry and analyst feedback while retaining the current explainability, safety boundary, and rule-based fallback.

Source of truth: the MIRAGE-X repository README, backend modules, frontend dashboard, Docker configuration, build guide, and automated tests.